

Example 1: Single Loop

- Code:
 - `for(i=0;i<n;i++)`
 - `printf("Hello");`
- Time Complexity: $O(n)$
- Explanation: Loop runs n times.

Example 2: Constant Work in Loop

- Code:
- `for(i=0;i<n;i++)`
- `a=b+c;`
- Time Complexity: $O(n)$
- Explanation: Constant work n times.

Example 3: Two Independent Loops

- Two loops each run n times.
- Time Complexity: $O(n)$
- Explanation: $n + n = 2n \rightarrow O(n)$.
(constant 2 is ignored)

```
for(i = 0; i < n; i++)  
{  
    printf("A");  
}  
  
for(j = 0; j < n; j++)  
{  
    printf("B");  
}
```

Example 4: Nested Loops

- Outer loop n times
- Inner loop n times
 - Outer loop runs n times
 - Inner loop runs n times for each outer loop
 - Total operations = $n \times n$
- Time Complexity: $O(n^2)$.

```
for(i = 0; i < n; i++)  
{  
    for(j = 0; j < n; j++)  
    {  
        printf("*");  
    }  
}
```

Example 5: Triangular Loop

- Inner loop runs 1, 2, 3, ... n times
- Total operations = $\frac{n(n+1)}{2}$
- Time Complexity: $O(n^2)$.

```
for(i = 0; i < n; i++)  
{  
    for(j = 0; j <= i; j++)  
    {  
        printf("*");  
    }  
}
```

Example 6: Increment by 2

- Loop runs $n/2$ times

```
for(i = 0; i < n; i += 2)
{
    printf("Hello");
}
```

- Loop runs $n/2$ times
- Constants are ignored

How does i change?

- Starts at 1
- Then: 3, 5, 7, 9, ...
- Increases by **2 each time**

Number of iterations

The loop runs until $i \leq n$

So roughly:

$n/2$ iterations

Time Complexity

- Constants are ignored in Big-O
- $n/2$ becomes **$O(n)$**

- Time Complexity: $O(n)$.

Example 7: Logarithmic Loop

- i doubles each time

```
for(i = 1; i <= n; i = i * 2)
{
    printf("Hello");
}
```

- i doubles every iteration
- Number of iterations = $\log_2 n$

Time Complexity

Number of iterations = $\log_2 n$

Time Complexity = $O(\log n)$

How does i change?

- Starts at 1
- Then: 2, 4, 8, 16, 32, ...
- Multiplies by **2 each time**

Values of i

After k iterations:

$$i = 2^k$$

Loop stops when:

$$2^k > n$$

Taking log:

$$k = \log_2 n$$

Example 8: Binary Search

- Search space halves each time

```
while(low <= high)
{
    mid = (low + high) / 2;

    if(key == a[mid])
        break;
    else if(key < a[mid])
        high = mid - 1;
    else
        low = mid + 1;
}
```

- Array size reduces to half each step
- Fast searching method
- Time Complexity: $O(\log n)$.

Example 9: $n \log n$

- Outer loop n
- Inner loop $\log n$

```
for(i = 1; i <= n; i++)  
{  
    for(j = 1; j <= n; j = j * 2)  
    {  
        printf("*");  
    }  
}
```

- Time Complexity: $O(n \log n)$.

Example 10: Constant Time

- Single statement

```
printf("Hello World");
```

- Executes only once
 - No dependency on input size
- Time Complexity: $O(1)$.

$O(n!)$

- **Example: Arranging Students in a Line**
- **Problem**
- Suppose there are **n students** and you want to find **all possible seating arrangements** in a straight line.
- Think of **3 students: A, B, C**
- Possible arrangements:
 - ABC
 - ACB
 - BAC
 - BCA
 - CAB
 - CBA

Total = $3! = 6$ arrangements

C programs to check a prime number using a User-Defined Function (UDF).

Each method uses a **different logic**

Method 1: Normal Approach (Check till $n-1$)

```
int isPrime(int n) {  
    int i;  
  
    if (n <= 1)  
        return 0;  
  
    for (i = 2; i < n; i++) {  
        if (n % i == 0)  
            return 0;  
    }  
    return 1;  
}
```

How Many Times Does the Loop Run?

- In the **worst case**, when n is prime
→ loop runs from 2 to $n-1$
→ total iterations $\approx n - 2$

Time Complexity

- **Worst Case:** $O(n)$
- **Best Case:** $O(1)$
(if n is divisible by 2, loop stops immediately)
- **Average Case:** $O(n)$

For number 13 :

for ($i = 2; i < 13; i++$)

Values of i

2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

Number of iterations

11 times

Method 2: Optimized Approach (Check till $n/2$)

```
int isPrime(int n) {  
    int i;  
  
    if (n <= 1)  
        return 0;  
  
    for (i = 2; i <= n / 2; i++) {  
        if (n % i == 0)  
            return 0;  
    }  
    return 1;  
}
```

How Many Times Does the Loop Run?

- In the **worst case**, when n is prime
→ loop runs approximately $n/2$ times

Time Complexity Calculation

$n/2$ → ignore constant $1/2$

Time Complexity

- **Worst Case:** $O(n)$
- **Best Case:** $O(1)$
- **Average Case:** $O(n)$

Explanation (Exam Line)

Although the loop runs up to $n/2$, constants are ignored in time complexity, so it is still **$O(n)$** .

for ($i = 2$; $i \leq 13/2$; $i++$) $13/2 = 6$

Values of i

2, 3, 4, 5, 6

Number of iterations

5 times

Method 3: Best Optimized Approach (Check till $\sqrt{n}$)

```
int isPrime(int n) {  
    int i;  
  
    if (n <= 1)  
        return 0;  
  
    for (i = 2; i * i <= n; i++) {  
        if (n % i == 0)  
            return 0;  
    }  
    return 1;  
}
```

- The program runs only $\sqrt{n}$ **times**
- Growth is **much slower**

If:

- $n = 100 \rightarrow 10$ steps
- $n = 10,000 \rightarrow 100$ steps

Time Complexity

- Worst Case:** $O(\sqrt{n})$
- Best Case:** $O(1)$
- Average Case:** $O(\sqrt{n})$

for (i = 2; i * i <= 13; i++)

$\sqrt{13} \approx 3.6$

Values of i

2, 3

- $2 \times 2 = 4 \leq 13$ ✓
- $3 \times 3 = 9 \leq 13$ ✓
- $4 \times 4 = 16 > 13$ ✗ $\rightarrow$ stop

Number of iterations

2 times

For number **13**, the $\sqrt{n}$ method checks divisibility only **2 times**, making it the most efficient.

- **Finding a factor of a number:**
- **$O(n)$:** Check **every number** from 1 to n
- **$O(\sqrt{n})$:** Stop checking at $\sqrt{n}$ because factors repeat after that
- ✓ Same result
 - ✓ Less work
 - ✓ Faster

“Time complexity tells us how fast the number of operations grows as input size n increases.”

What is *input size* (n)?

- n represents **how much data** the program is working on
- Examples:
 - Number of elements in an array
 - Value of a number
 - Number of nodes in a tree

Example:

If an array has 10 elements $\rightarrow n = 10$

If it has 1,000 elements $\rightarrow n = 1000$

What are *operations*?

Operations are the **basic steps** a program performs, such as:

- Comparisons
- Loop iterations
- Arithmetic calculations

Example:

```
for(i = 1; i <= n; i++)  
{  
    printf("Hello");  
}
```

Here, printf is executed **n times** $\rightarrow n$ operations

What does *grows* mean?

It means:

- How the **number of steps increases**
- When n becomes larger

Example:

n	Number of operations
10	10
100	100
1000	1000

Operations increase as n increases

What does *how fast* mean?

It compares **rate of increase**, not exact time.

Compare two programs:

Program A ($O(n)$)

- Operations increase linearly

Program B ($O(n^2)$)

- Operations increase very fast

n	$O(n)$	$O(n^2)$
10	10	100
100	100	10,000

Program A is **faster** because its operations grow slowly

Suppose you are checking notebooks:

- $O(n)$: Check notebooks one by one
- $O(n^2)$: Check every notebook with every other notebook

As notebooks increase, the second method becomes **very slow**

- Time complexity describes how the number of operations performed by an algorithm increases as the input size n increases.
- Time complexity is about **growth**, not exact time
 - ✓ It helps compare algorithms
 - ✓ Slower growth = better algorithm